1.0 - Operative system

What's an OS

- Resource allocator - manages cpu and
- Controller

Multiprogramming - Multitasking / Timeslicing

- Operative sys switches from different process to keep the CPU busy
- Utilizes timeslicing to incrementally progress on all processes
- Linux is the 4th generation of operative systems (1991).
- 5th gen - Virtualization: VMs hypervizors and containers

Kernel structures

- Monolithic - linux - runs everything - fast - vulnerable
- Hybrid - Windows - microkernel wrapped in monolithic layers.

Operating system structure:

User applications	Your own programs
System call interface	your programs access portal to system
Kernel	controller of system programs/services
System programs/services	File system, memory management, I/O, etc

Dual mode operation:

- Usermode and kernel mode - mode bit 1/0
- separates the userprocesses from the kernel

1.1 - Processes

Process

- Is an active entity living in main memory
- Memory layout
    - Text section - The program's executable code
    - Data section - The global variables
    - Heap section - dynamically allocated memory - runtime
    - Stack section - Temporary data storage - registers local variables etc
    - The heap grows towards the stack vice versa.

Process Control Block
Information regarding the process state - needed by the operating system.

- Process state.
- Process number
- Program counter
- Registers
- Open files

Process state:

- New - The Process is being created
    - Admitted: New => Ready

- Ready - Process is waiting to be assigned CPU compute
    - Scheduler dispatch: Ready => Running

- Running - Process is executing
    - Interrupt: Running => Ready
    - I/O | Wait: Running => Waiting
    - Exit: Running => Terminated

- Waiting - Process is waiting for an event such as I/O
    - I/O | Event completion: Waiting => Ready

- Terminated - Process is finished

Process Creation - Parent & Children

Fork()

- creates new Process
- PID = Process ID, number
- PPID = parent Process ID
    - Parent gets returned children id from fork()
    - child gets returned 0 from fork()

Fork () => (Parent PID > 0, Child PID == 0)

- Parent can call Wait() to wait for the child to call Exit()
- If a child calls exec() it creates a new process as a new fresh process

Clone()

- Similar to fork(), but with more control over memory space.

Process states

Zombie - A child has terminated but the parent is not yet ware

Orphan - The parent has terminated - but the child lives

Process scheduling

CPU Utilization

- Time CPU is used / Total time

Multiprogramming

- Switching between process to improve CPU utilization
- Could be due too I/O and starvation

Context switch

- Multiprogramming comes at a cost => context switching. Switching between processes has an overhead cost which is the time to prepare for a new process to start computing. Ex : saving data and loading data

Inter-Process Communication (IPC)

- IPC allows for processes to share data
- Shared memory
    - Data is shared in a shared memory space - often the fastest.
    - mmap(), shm_open(), shm_unlink().
- No synchronisation
- Message passing
    - Datastreams
    - Pipe
        - Gives two process a consumer producer style of communication
        - Requires parent - child relation. read from pipe[0] write to pipe[1]
    - FIFO (named pipe)
        - allows multiprocess to communicate - does not require parent child relation
        - open(), read(), write(), and close() system calls
    - Message queue

1.2 Threads

Treads VS Processes

Threads share the executable code data and files - but keeps own registers and stacks.

Scheduling for threads

- Threads are subject to scheduling similar to processes.
- On multicore CPUs threads are distributor on many cores for better performance

User Threads

- Managed by the user / user programs
- Ex: Posix pthreads, java threads

Kernel Threads

- Managed by the kernel - to improve efficiency
- Many to one
    - Uncommon today: blocks - long sys calls.
- One to one
    - one user thead = one kernel thread.
    - more concurrency - limited to overhead costs
- One to many
    - Many user threads map to another amount of kernel threads
    - allows for balanced amount of kernel threads

Concurrency

Allows for execution of multiple tasks but not at the same time

- A scheduler provides concurrency

Parallellism

Execution on multiple tasks at the same time

- Multicore provides parallelism
- Data parallelism
    - Splits the data into separate chunks that can be computed simultaneously
- Task Parallelism
    - Perform different tasks simultaneously on the same data

Thread Pools

- Creating a pool of threads that wait for tasks since creating threads has a large overhead.